

MecâniQA Nuvem - Time Vitoria

Autores: Aeverton Santos de Oliveira
Caio Victor Lopes Serra
Carlos Henrique Santos de Carvalho
Jonatas de Jesus Souza
Maciel Lopes Francisco
Natanael de Jesus Noronha

Itabuna, 2026

Agenda

1. Contexto:

Objetivo do projeto e arquitetura da aplicação

2. Containerização:

Dockerfiles e execução dos serviços

3. Docker Compose:

Orquestração local da API, MySQL e Redis

4. Kubernetes:

Deployments, Pods, Services e ConfigMap

5. Observabilidade:

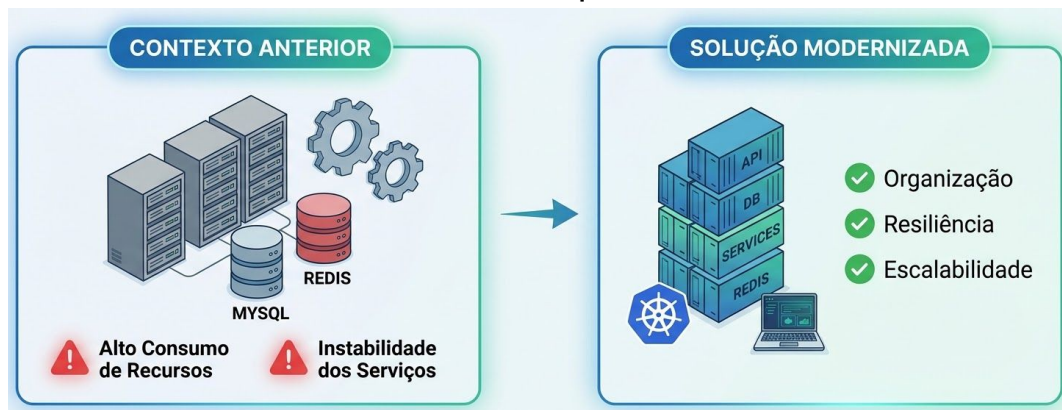
K9s e teste de auto-recuperação

6. Infrastructure as code:

Terraform e gerenciamento declarativo

Contexto do Problema

- A MecâniQA é uma startup SaaS voltada para oficinas mecânicas. A aplicação é composta por uma API Java/Spring Boot, MySQL e Redis.
- A infraestrutura anterior apresentava desafios relacionados ao consumo de recursos e à estabilidade dos serviços.
- Objetivo do projeto: estruturar uma base de infraestrutura mais organizada e resiliente utilizando containers e orquestração.
- Entregável: serviços containerizados, ambiente local orquestrado e recursos Kubernetes definidos de forma declarativa.



MecâniQA

Arquitetura e Estrutura dos Serviços



Java / Spring boot

API principal
Porta: 8080
Java 17

MySql

Banco de dados transacional
Porta: 3306
MySQL 8.0

Redis

Cache / Armazenamento em Memória
Porta: 6379
Redis 7 Alpine

- Cada serviço possui seu próprio Dockerfile e imagem independente.
- A comunicação entre os serviços é realizada por nomes de serviço, evitando dependência de IPs fixos.
- O MySQL utiliza volume nomeado para persistência dos dados.

Docker Compose



- O docker-compose.yml reúne API Java, MySQL e Redis em uma única aplicação.
- Os serviços compartilham uma rede interna criada pelo Docker Compose.
- A API acessa o banco utilizando o hostname do serviço: db.
- O MySQL utiliza o volume db_data:/var/lib/mysql para preservar os dados.
- Comandos principais: docker compose up -d, docker compose ps e docker compose down.

```
docker-compose.yml x
services:
  db:
    build:
      context: .
      dockerfile: mysql/Dockerfile
    environment:
      MYSQL_ROOT_PASSWORD: 123456
    volumes:
      - db_data:/var/lib/mysql

  redis:
    build:
      context: .
      dockerfile: redis/Dockerfile

  api:
    build:
      context: .
      dockerfile: java/Dockerfile
    ports:
      - "8080:8080"

volumes:
  db_data:
```

Docker - Isolamento dos serviços



- Criamos imagens independentes para a API Java, MySQL e Redis.
- Cada serviço possui seu próprio container.
- Utilizamos Dockerfiles para definir como cada imagem é construída.

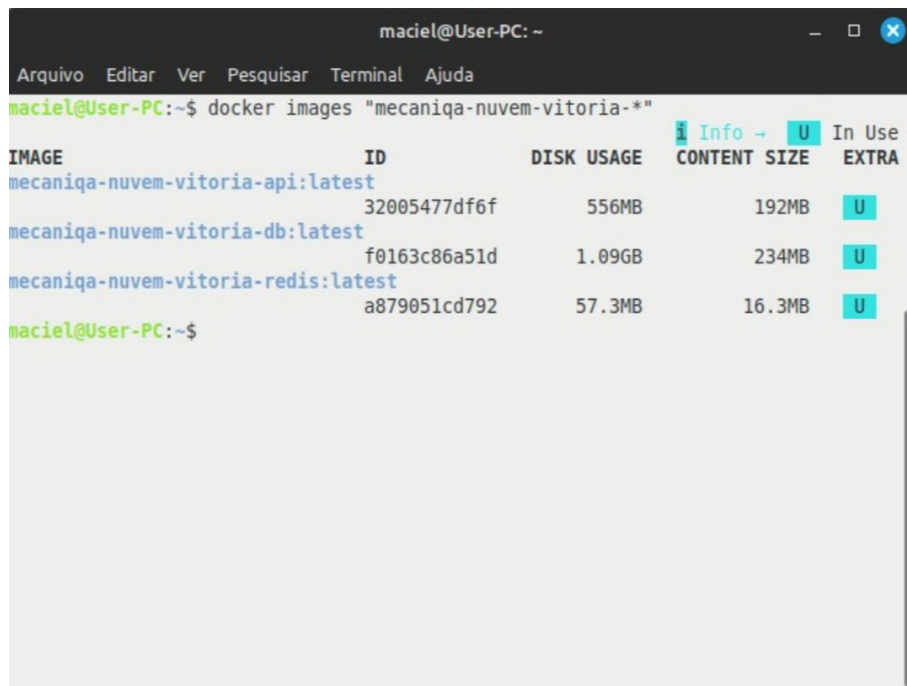
A screenshot of a terminal window titled 'maciel@User-PC: ~'. The terminal shows the command 'docker images "mecanica-nuvem-vitoria-*"' and its output. The output is a table with columns: IMAGE, ID, DISK USAGE, CONTENT SIZE, and an 'In Use' status. The table lists three Docker images: 'mecanica-nuvem-vitoria-api:latest', 'mecanica-nuvem-vitoria-db:latest', and 'mecanica-nuvem-vitoria-redis:latest'. Each image has a corresponding ID, disk usage, content size, and a blue 'U' icon indicating it is in use. The terminal window has a menu bar with options: Arquivo, Editar, Ver, Pesquisar, Terminal, Ajuda.

IMAGE	ID	DISK USAGE	CONTENT SIZE	In Use
mecanica-nuvem-vitoria-api:latest	32005477df6f	556MB	192MB	U
mecanica-nuvem-vitoria-db:latest	f0163c86a51d	1.09GB	234MB	U
mecanica-nuvem-vitoria-redis:latest	a879051cd792	57.3MB	16.3MB	U

Docker Compose - Orquestração local



- O Docker Compose permite executar os três serviços de forma integrada.
- API, MySQL e Redis são executados simultaneamente.
- Os serviços se comunicam pela rede interna do Compose.
- O MySQL utiliza volume para persistência dos dados.

```
maci@User-PC: ~/mecaniQA-nuvem-vitoria
Arquivo  Editar  Ver  Pesquisar  Terminal  Ajuda

maci@User-PC:~/mecaniQA-nuvem-vitoria$ docker compose ps
NAME                                IMAGE                                COMMAND
SERVICE    CREATED      STATUS      PORTS
mecaniqa-nuvem-vitoria-api-1        mecaniqa-nuvem-vitoria-api         "/_cacert_entry
poin..."  api         2 days ago  Up About a minute  0.0.0.0:8080->8080/tcp, [::]
:8080->8080/tcp
mecaniqa-nuvem-vitoria-db-1         mecaniqa-nuvem-vitoria-db         "docker-entrypoi
nt.s..."  db          4 days ago  Up About a minute  3306/tcp, 33060/tcp
mecaniqa-nuvem-vitoria-redis-1      mecaniqa-nuvem-vitoria-redis      "docker-entrypoi
nt.s..."  redis       4 days ago  Up About a minute  6379/tcp
maci@User-PC:~/mecaniQA-nuvem-vitoria$
```

K9s - Observabilidade

Utilizado para observar e administrar o cluster Kubernetes.

- Pods e Deployments
- Services
- Logs
- Estado dos containers
- Reinicializações

Os Pods permaneceram em estado Running durante a inspeção.

Terraform — Infrastructure as Code

Provider Kubernetes utilizado para definir declarativamente o namespace mecaniqua.

Fluxo utilizado:
terraform init
terraform plan
terraform apply

O plan também foi usado para verificar a convergência entre configuração e infraestrutura.

Kubernetes - Gerenciamento dos serviços



- Criamos um cluster Kubernetes local utilizando Kind.
- Os serviços foram convertidos para Deployments e Pods.
- A API possui 3 réplicas para aumentar a disponibilidade.
- MySQL e Redis possuem uma réplica cada.

A screenshot of a terminal window titled 'maciel@User-PC: ~/mecaniQA-nuvem-vitoria'. The terminal shows the command 'kubectl get pods' and its output, which is a table of pod information. The table has five columns: NAME, READY, STATUS, RESTARTS, and AGE. There are five rows of pod data, all showing a 'Running' status. The terminal window has a dark theme and standard window controls (minimize, maximize, close) in the top right corner.

```
maciel@User-PC: ~/mecaniQA-nuvem-vitoria$ kubectl get pods
```

NAME	READY	STATUS	RESTARTS	AGE
api-946dd6988-cqt25	1/1	Running	3 (3h55m ago)	21h
api-946dd6988-fw4t2	1/1	Running	3 (3h55m ago)	22h
api-946dd6988-rht5f	1/1	Running	3 (3h55m ago)	20h
db-565df6d748-m6kdt	1/1	Running	3 (3h55m ago)	20h
redis-765fdf6bf6-slcxb	1/1	Running	3 (3h55m ago)	21h

```
maciel@User-PC: ~/mecaniQA-nuvem-vitoria$
```

Kubernetes - Comunicação e exposição



- A API utiliza um NodePort para receber requisições externas.
- MySQL e Redis utilizam ClusterIP, permanecendo acessíveis apenas internamente.
- O Kubernetes fornece comunicação entre os serviços através de seus nomes.

```
maci@User-PC: ~/mecaniQA-nuvem-vitoria
Arquivo  Editar  Ver  Pesquisar  Terminal  Ajuda
maci@User-PC:~/mecaniQA-nuvem-vitoria$ kubectl get services
NAME      TYPE        CLUSTER-IP    EXTERNAL-IP  PORT(S)          AGE
api       NodePort    10.96.23.101  <none>       8080:30864/TCP   22h
db        ClusterIP   10.96.62.20   <none>       3306/TCP         21h
kubernetes ClusterIP   10.96.0.1     <none>       443/TCP          22h
redis     ClusterIP   10.96.127.92  <none>       6379/TCP         21h
maci@User-PC:~/mecaniQA-nuvem-vitoria$
```

K9s - Monitoramento do cluster



- Utilizamos o K9s para visualizar e gerenciar os recursos do Kubernetes.
- A ferramenta permite acompanhar Pods, Deployments e seus estados.
- Os serviços aparecem em execução dentro do cluster.

```
maci@User-PC: ~/mecaniQA-nuvem-vitoria
Arquivo  Editar  Ver  Pesquisar  Terminal  Ajuda

Context: kind-kind [RW]
Cluster: kind-kind
User:    kind-kind
K9s Rev: v0.51.0
K8s Rev: v1.37.0
CPU:     n/a
MEM:     n/a

<0>
<1>
K9S

pods(default)[5]
NAME          PF  READY  STATUS  RESTARTS  IP           NODE
api-946dd6988-cqt25  ●  1/1    Running    3  10.244.0.7  kind-control-
api-946dd6988-fw4t2  ●  1/1    Running    3  10.244.0.5  kind-control-
api-946dd6988-rht5f  ●  1/1    Running    3  10.244.0.8  kind-control-
db-565df6d748-m6kdt  ●  1/1    Running    3  10.244.0.9  kind-control-
redis-765fdf6bf6-slcxb  ●  1/1    Running    3  10.244.0.6  kind-control-

<pod>
```

Evolução da infraestrutura

- Isolamento dos serviços com containers.
- Orquestração local com Docker Compose.
- Gerenciamento e resiliência com Kubernetes.
- Monitoramento com K9s.
- Infraestrutura declarativa com Terraform.



Considerações Finais



- A infraestrutura evoluiu de containers individuais para uma arquitetura orquestrada com Docker Compose e Kubernetes.
- A aplicação passou a contar com comunicação interna por Services e recuperação automática de Pods.
- K9s facilitou a observação do estado do cluster e dos workloads.
- Terraform introduziu Infrastructure as Code para gerenciamento declarativo da infraestrutura.
- Próximos passos: adicionar persistência mais robusta no Kubernetes, Secrets, health checks, recursos/limites e evolução para um ambiente cloud.

Referências



- Docker — Documentação oficial: <https://docs.docker.com/>
- Kubernetes — Documentação oficial: <https://kubernetes.io/docs/>
- Kind — Documentação oficial: <https://kind.sigs.k8s.io/>
- K9s — Documentação oficial: <https://k9scli.io/>
- Terraform — HashiCorp: <https://developer.hashicorp.com/terraform>